TALENT OPTIMIZATION PLATFORM

PROJECT REPORT

Submitted by

SRI RAGUL A A (7376231EC290)

In partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING



BANNARI AMMAN INSTITUTE OF TECHNOLOGY
(An Autonomous Institution Affiliated to Anna University, Chennai) SATHYAMANGALAM-638401

ANNA UNIVERSITY: CHENNAI 600 025

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project report **"TALENT OPTIMIZATION PLATFORM"** is the Bonafide work of **"SRI RAGUL A A (7376231EC290)"** who carried out the project work under my supervision.

Dr. PRAKASH S P
HEAD OF THE DEPARTMENT
Department of Electronics and
Communication Engineering
Bannari Amman Institute of Technology

Dr. STEPHEN SAGAYARAJ A
ASSISTANT PROFESSOR III
Department of Electronics and
Communication Engineering
Bannari Amman Institute of Technology

Submitted for Project Viva Voice examination held on. 2/04/2026

Internal Examiner I

Internal Examiner II

DECLARATION

I affirm that the project work titled **"TALENT OPTIMIZATION PLATFORM"** being submitted in partial fulfilment for the award of the degree of **Bachelor of Engineering** in **Electronics and Communication Engineering** is the record of original work done by us under the guidance of **Dr. STEPHEN SAGAYARAJ A**, Assistant Professor III, Department of Electronics and Communication Engineering It has not formed a part of any other project work(s) submitted for the award of any degree or diploma, either in this or any other University.

Sri Ragul A A

SRI RAGUL A A

(7376231EC290)

I certify that the declaration made above by the candidate is true.

Dr. STEPHEN SAGAYARAJ A

ACKNOWLEDGEMENT

We would like to enunciate heartfelt thanks to our esteemed Chairman **Dr. S.V. Balasubramaniam**, and the respected Principal **Dr. C. Palanisamy** for providing excellent facilities and support during the course of study in this institute.

We are grateful to **Dr. Prakash S P, Associate Professor and Head of the Department, Department of Electronics and Communication Engineering** for his valuable suggestions to carry out the project work successfully.

We wish to express our sincere thanks to Faculty guide **Dr. Stephen Sagayaraj A, Assistant Professor III, Department of Electronics and Communication,** for his constructive ideas, inspirations, encouragement, excellent guidance, and much needed technical support extended to complete our project work.

We would like to thank our friends, faculty and non-teaching staff who have directly and indirectly contributed to the success of this project.

Sri Ragul A A

SRI RAGUL A A(231EC290)

ABSTRACT

This project presents a Talent Optimization Platform (Skill Bridge**)** a full-stack, career intelligence web application designed to bridge the gap between student talent and employer demand. The platform provides students with intelligent tools to manage their skills, projects, and certifications, while computing a Readiness Score that quantifies their suitability for specific technical roles such as Full Stack Developer, Data Scientist, ML Engineer, or DevOps Engineer. The system analyses each student's skill proficiency levels, project portfolio, certifications, academic CGPA, and profile completeness against role-specific requirement matrices. A weighted scoring engine assigns 40 points for skills, 20 for projects, 15 for certifications, 15 for profile completeness, and 10 for CGPA producing a transparent and motivating score out of 100. The platform also provides a Skill Gap Analysis that identifies missing core and supplementary skills for any chosen target role, along with a personalized Action Plan containing phased, resource-linked learning steps.

Employers are provided a dedicated portal to search and filter student talent by skill, university, CGPA, department, and readiness score. The employer dashboard presents platform-wide analytics including top skills in demand, CGPA distributions, and student activity trends.

The platform is built on the MERN stack MongoDB, Express.js, React.js, and Node.js with TailwindCSS for responsive design, Recharts for interactive data visualization, and deployed on Render and Vercel for cloud hosting. Skill Bridge supports dark mode, PDF resume generation, and mobile-responsive layouts, delivering a comprehensive, data-driven solution for talent optimization in educational institutions.

KEYWORDS: Talent Optimization, Skill Gap Analysis, Readiness Score, MERN Stack, Employer Search, MongoDB, React.js, JWT Authentication

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	BONAFIDE CERTIFICATE	**I**
	DECLARATION	**II**
	ACKNOWLEDGEMENT	**III**
	ABSTRACT	**IV**
	TABLE OF CONTENT	**V**
	LIST OF FIGURES	**IX**
	LIST OF TABLES	**X**
1	**INTRODUCTION**	**01**
	1.1 OUTLINE	01
	1.2 SCOPE OF THE PROJECT	01
	1.3 SUMMARY	04
2	**LITERATURE SURVEY**	**05**
	2.1 OUTLINE	05
	2.2 EXISTING WORKS	05
	2.3 LIMITATIONS OF EXISTING WORKS	09
	2.4 SUMMARY	09
3	**OBJECTIVES AND METHODOLOGY**	**10**

CHAPTER NO	TITLE	PAGE NO
	3.1 OUTLINE	10
	3.2 OBJECTIVES OF THE PROPOSED SOLUTION	10
	3.3 FLOW DIAGRAM	11
	3.3.1 STUDENT DESIGN PANEL	12
	3.3.2 EMPLOYER DESIGN PANEL	13
	3.4 SELECTION OF COMPONENTS, TOOL AND TECHNIQUES	13
	3.4.1 FRONT END DEVELOPMENT	13
	3.4.2 BACKEND DEVELOPMENT	14
	3.4.3 DATABASE SELECTION	14
	3.4.4 READINESS ENGINE	15
	3.4.5 MATHEMATICAL MODEL OF READINESS SCORE	
	3.4.6 AUTHENTICATION AND SECURITY	17
	3.5 SUMMARY	17
4	**PROPOSED WORK MODULES**	**18**
	4.1 OUTLINE	18
	4.2 PROPOSED WORK	19
	4.2.1 USER AUTHENTICATION MODULE	19
	4.2.2 STUDENT PROFILE MODULE	19
	4.2.3 SKILL AND PROJECT MANAGEMENT MODULE	20

CHAPTER NO	TITLE	PAGE NO
	4.2.4 READINESS SCORE AND SKILL GAP MODULE	20
	4.2.5 ACTION PLAN AND RECOMMENDATION MODULE	21
	4.2.6 EMPLOYER TALENT SEARCH MODULE	23
	4.3 METHODOLOGY OF THE PROPOSED WORK	24
	4.4 SUMMARY	26
5	**RESULTS AND DISCUSSION**	**27**
	5.1 INTRODUCTION	27
	5.2 RESULTS	27
	5.2.1 USER AUTHENTICATION RESULTS	27
	5.2.2 STUDENT DASHBOARD RESULTS	28
	5.2.3 READINESS SCORE RESULTS	29
	5.2.4 EMPLOYER SEARCH RESULTS	30
	5.2.5 DATABASE AND API PERFORMANCE	31
	5.2.6 COMPARISON WITH RELATED WORKS	32
	5.3 SIGNIFICANCE, STRENGTHS, AND LIMITATIONS OF PROPOSED WORK	32
	5.4 COST-BENEFIT ANALYSIS	34
	5.5 SUMMARY	35
6	**CONCLUSION AND FUTURE WORK**	**36**
	6.1 CONCLUSION	36

CHAPTER NO	TITLE	PAGE NO
	6.2 SUGGESTIONS FOR FUTURE WORK	37
	6.3 SUMMARY	39
7	**REFERENCES**	**40**
8	**APPENDICES**	**42**

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
3.1	STUDENT PANEL AND EMPLOYER FLOW DIAGRAM	11
4.1	SKILLBRIDGE SYSTEM ARCHITECTURE DIAGRAM	18
5.1	LOGIN PAGE	28
5.2	STUDENT DASHBOARD WITH CHARTS	29
5.3	READINESS SCORE AND ACTION PLAN	30
5.4	EMPLOYER TALENT SEARCH PAGE	31

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
5.1	COMPARATIVE ANALYSIS BETWEEN SKILLBRIDGE AND EXISTING SYSTEMS	32
5.2	COST-BENEFIT SUMMARY FOR SKILLBRIDGE	34

CHAPTER I

INTRODUCTION

1.1 OUTLINE

The Talent Optimization Platform, referred to throughout this report as Skill Bridge, is a web-based career intelligence system that serves two distinct user groups students who want to understand and improve their readiness for specific technical roles, and employers who want to find and evaluate candidates efficiently. The platform bridges the gap between what students can demonstrate and what employers need to see, replacing informal self-assessment and unstructured resumes with a transparent, data-driven scoring model and a structured, searchable talent directory.

Students build digital profiles on Skill Bridge by registering their skills with proficiency ratings, adding their projects with technology stacks, logging certifications, and completing academic and personal details. The platform uses this structured data to compute a Readiness Score out of one hundred for any career role the student targets. Employers log into a separate portal, search the student database with multiple filters, and receive score-ranked results without any manual screening effort. The system was built to make career readiness measurable and talent discovery efficient, using the MERN stack deployed on free cloud infrastructure.

1.2 SCOPE OF THE PROJECT

To fully understand the scope of Skill Bridge, it helps to first appreciate the environment it was designed for. The higher education system in India produces a very large number of engineering and technology graduates each year. A significant portion of these graduates are technically capable people who have learned programming, built projects, and earned certifications during their studies. However, when they enter the job market, they often struggle to communicate their readiness in a way that resonates with

employers. Traditional resumes do not convey proficiency levels, do not show how a student's skill set maps to a specific role's requirements, and do not give any indication of what gaps still exist. Employers, meanwhile, have developed their own imperfect workarounds standardized entrance tests, group interviews, and lengthy manual screening processes all of which add cost and time without necessarily producing better hiring outcomes.

The scope of Skill Bridge is to provide a platform that addresses this problem end to end. On the student side, the platform covers every aspect of a student's professional profile. Students can register accounts, choose their role as a student, and begin building their profile immediately. They can add technical skills one by one, specifying not just the name of the skill but the category it belongs to, such as Frontend, Backend, Data Science, DevOps, or Mobile, and their honest proficiency level on a five-point scale from Beginner to Master. The system checks for duplicate skill entries so that students cannot accidentally add the same skill twice with different spellings. The academic and personal section of the profile captures CGPA, university name, department, age, and professional links to GitHub and LinkedIn, along with a short bio. The platform tracks how complete this section is and incorporates profile completeness into the readiness score, incentivising students to provide thorough and accurate information.

The Readiness Engine, which sits at the heart of the platform's intelligence, currently covers twelve or more distinct technical roles. These include Full Stack Developer, Frontend Developer, Backend Developer, Data Scientist, Machine Learning Engineer, Data Engineer, DevOps Engineer, Mobile Developer, Android Developer, iOS Developer, UI/UX Designer, and Cybersecurity Analyst. Each role has its own requirement matrix that classifies needed skills into three tiers: core skills that are essential and carry the highest scoring weight, strong skills that are expected by most employers for that role, and bonus skills that differentiate candidates and push scores higher without being mandatory.

The Skill Gap Analysis feature uses this same role-requirement matrix to generate a visual comparison between what the student has and what the role demands. Missing core skills are flagged with a High severity indicator, missing strong skills are flagged as medium severity, and missing bonus skills are flagged as Low. For each missing skill, the platform provides a direct link to the most appropriate official learning resource, whether that is official documentation, a popular tutorial platform, or an open-source guide.

The Action Plan module builds on the Skill Gap Analysis by organizing the recommended improvements into a structured, time-ordered set of phases. This turns the raw gap data into something immediately actionable rather than just informational. Phase One focuses on acquiring the most critical missing core skills. Phase Two shifts the focus to building projects that demonstrate those skills in practice. Phase Three covers certifications and profile polish. Each action item in the plan is tagged with the type of effort required and includes resource links so the student does not have to go hunting for learning materials on their own.

On the employer side, the scope extends to a full search and discovery experience. Employers can search the student database using any combination of a keyword query, a specific skill name, a minimum CGPA, a university filter, a department filter, and a target role. The system applies all active filters simultaneously and returns a paginated list of matching students. For each result, the readiness score for the specified target role is computed on the fly so that the scores employers see are always based on the student's most current profile data. Results can be sorted either by readiness score or by CGPA depending on what the employer prioritises. Each student card in the results shows the student's name, university, department, CGPA, top skills, project count, and readiness score, giving employers enough information to make a quick initial assessment before clicking through to the full profile.

The technical scope of the project encompasses the full MERN stack. The frontend is a React 18 application built with Vite, styled with TailwindCSS, and uses Recharts for

all data visualizations and Axios for backend communication. The backend is a Node.js and Express.js application that exposes over thirty RESTful API endpoints across three route groups authentication, student, and employer. Data is stored in MongoDB Atlas. The application is deployed on Render for the backend service and Vercel for the frontend, both running on free-tier plans to keep the infrastructure cost at zero.

1.3 SUMMARY

This chapter introduced the Talent Optimization Platform and established the core problem it solves the lack of structured, objective career readiness measurement for students and the inefficiency of manual candidate screening for employers. Skill Bridge addresses both sides by combining a multi-factor readiness scoring engine, a skill gap analysis tool, a phased action plan, and an employer search portal into a single zero-cost platform built on the MERN stack. The following chapters detail the existing research in this area, the platform's objectives and methodology, its modules, and the results from testing.

CHAPTER II

LITERATURE SURVEY

2.1 OUTLINE

Research into digital tools for skill assessment, career readiness, and talent management has grown steadily alongside the expansion of online education and the increasing complexity of technical hiring. This chapter reviews seven works that are directly relevant to Skill Bridge, covering competency-based profiling, machine learning skill gap identification, automated resume screening, educational recommender systems, portfolio platforms, MERN stack development, and weighted employability scoring. The gaps left by these works define the contribution of the proposed system.

2.2 EXISTING WORKS

2.2.1 Competency-Based Student Profile Systems

Hernandez et al. [1] proposed a competency-based academic profiling system that mapped student skills to industry-standard frameworks and let institutions track progress against defined competency levels. The system improved employer-institution engagement and motivated students to address identified weaknesses during their studies. However, the competency framework was institution-defined and static, with no employer search portal, no readiness scoring, and no personalized gap analysis or action planning. Students had no way to self-direct their profile beyond the prescribed domains.

2.2.2 Skill Gap Analysis Using Machine Learning

Roy and Bhatt [2] used machine learning to identify skill gaps in engineering students by comparing profiles against job descriptions scraped from online portals, applying TF-IDF vectorization and cosine similarity for matching. The system usefully revealed that student skill profiles often did not align with what employers were actually

posting for. Its limitations were significant for a student-facing platform, though it required continuous re-scraping to stay current, had no interactive student interface, provided no individual readiness scores, and had no employer-facing component connecting the analysis to real hiring.

2.2.3 Resume Parsing and Automated Screening

Chen et al. [3] built an NLP-based resume parsing system that extracted structured data from freeform resumes and ranked candidates against job descriptions, reducing recruiter screening time by approximately forty percent in controlled experiments. The fundamental limitation was its dependence on the resume as a data source — a student could list any skill regardless of proficiency, and the system had no way to distinguish depth from surface familiarity. Students also received no feedback and could not see how they were being scored, meaning the system provided no improvement pathway.

2.2.4 Educational Recommender Systems

Drachsler et al. [4] surveyed recommender systems in technology-enhanced learning, covering collaborative filtering, content-based filtering, and hybrid approaches over roughly a decade of research. The consistent finding was that recommender effectiveness dropped sharply when recommendations were not connected to specific, near-term learning goals the learner was motivated to pursue. Systems that framed recommendations as steps toward a defined career goal had significantly higher engagement than those that recommended resources in isolation. This finding directly informed Skill Bridge's approach of anchoring all recommendations to the student's chosen target role and readiness score.

2.2.5 Portfolio-Based Career Platforms

Iqbal et al. [5] found that students who maintained active digital portfolios were significantly more likely to secure employment within six months of graduation than those

who did not, with employers reporting that portfolios made initial screening substantially easier. However, most platforms evaluated were passive repositories with no analytical layer no readiness scoring, no gap analysis, no employer search, and no guidance on what to add next. The study concluded that intelligence built into the platform, rather than passive storage, was the key to improving outcomes for both parties.

2.2.6 MERN Stack Web Application Frameworks

Patel et al. [6] evaluated the MERN stack against several alternative architectures for data-intensive web applications, finding that MongoDB's schema flexibility handled rapidly evolving data models well, Node.js's non-blocking I/O gave a strong performance advantage under concurrent I/O-bound workloads, and React's component model accelerated development of complex stateful interfaces. JWT-based stateless authentication and RESTful API design were recommended as best practices for MERN applications supporting multiple client types — both of which were adopted directly in Skill Bridge.

2.2.7 Weighted Scoring for Career Readiness

Kumar and Singh [7] proposed a multi-factor weighted scoring model for engineering graduate employability incorporating academic performance, technical skill breadth, project portfolio depth, extracurricular involvement, and communication ability. The model had significantly higher correlation with actual employment outcomes than GPA alone. Critically, they also found that making the scoring model transparent to students — so they could see each component's contribution — produced better improvement outcomes than opaque systems, because students who understood the model took more targeted and effective action. Skill Bridge's Readiness Engine was designed with this finding as a core principle.

2.2.8 AI-Based Career Recommendation System

Kumar and Singh [8] proposed an AI-based recommendation system that suggests suitable career paths for students based on their skills and interests. The system improves decision-making but lacks integration with real-time employer requirements.

2.2.9 Skill Matching using Deep Learning

Sharma and Gupta [9] proposed a deep learning-based system for matching candidate skills with job requirements. The model improves matching accuracy by learning complex patterns in skill data. However, it requires large datasets for training and lacks transparency in decision-making for users.

2.2.10 Smart Job Portal Systems

Verma et al. [10] developed a smart job portal using machine learning algorithms to recommend suitable candidates for job roles. The system enhances recruitment efficiency but does not provide personalized feedback or readiness evaluation for students.

2.2.11 Student Skill Analysis Systems

Reddy and Rao [12] designed a web-based system to analyse student skills and provide career guidance. The system helps students understand their strengths but lacks dynamic role-based scoring and real-time industry alignment.

2.2.12 Career Recommendation Systems

Nguyen et al. [14] proposed a hybrid recommender system for career path planning. The system combines multiple recommendation techniques to improve accuracy. However, it does not integrate employer requirements or provide actionable skill gap analysis.

2.3 LIMITATIONS OF EXISTING WORKS

The reviewed works share a consistent pattern of limitations that Skill Bridge is designed to address. No single existing system combines structured, proficiency-rated skill registration; multi-factor role-specific readiness scoring; visual gap analysis with severity classification and resource links; phased personalized action planning; dynamic AI-style recommendations; employer search and filter with on-the-fly score computation; platform-wide analytics for employer context; PDF resume generation; and peer benchmarking — all in a single zero-cost platform. Most systems address one or two capabilities in isolation. The gap analysis systems lack employer portals. The portfolio platforms lack readiness scoring. The recommender systems are not connected to real hiring processes. Skill Bridge integrates all of these into one cohesive experience.

2.4 SUMMARY

The literature establishes strong evidence base for Skill Bridge's approach. Structured profiling outperforms unstructured resumes, transparent scoring models produce better student improvement outcomes, recommendations anchored to career goals outperform general resource suggestions, and employer-facing search substantially improves platform utility. The specific combination of features in Skill Bridge has not been implemented together in any reviewed prior work, confirming it as a genuine contribution to this problem space.

CHAPTER III

OBJECTIVES AND METHODOLOGY

3.1 OUTLINE

This chapter describes the objectives and development methodology of the Talent Optimization Platform. Serving two very different user groups — students managing career development and employers searching for candidates — requires careful attention to both the user experience of each group and the underlying architecture that makes both experiences possible simultaneously. This chapter makes the platform's goals explicit and describes how the development process was organized to achieve them.

3.2 OBJECTIVES OF THE PROPOSED SOLUTION

- To develop a data-driven Talent Optimization Platform that bridges the gap between student skills and employer requirements.
- To create a centralized system for students to manage and showcase their skills, projects, certifications, and academic details with proficiency levels.
- To design and implement a Readiness Engine that evaluates and computes a readiness score out of 100 for multiple technical roles.
- To perform role-specific Skill Gap Analysis by comparing student profiles with predefined role requirements and identifying missing skills.
- To generate a structured Action Plan that provides prioritized steps for improving skills, projects, certifications, and profile completeness.
- To develop an intelligent Recommendation Engine that provides dynamic and personalized suggestions based on the student’s profile and target role.
- To enable benchmarking of student performance by comparing individual metrics with platform-wide averages.

- To provide a feature for generating downloadable and professional PDF resumes from structured profile data.
- To build an employer portal that allows efficient searching, filtering, and evaluation of candidates based on multiple parameters such as skills, CGPA, and readiness score.
- To implement secure authentication using JWT with role-based access control for both students and employers.
- To design a responsive and user-friendly interface with support for modern UI features such as dark mode across multiple devices.

3.3 FLOW DIAGRAM

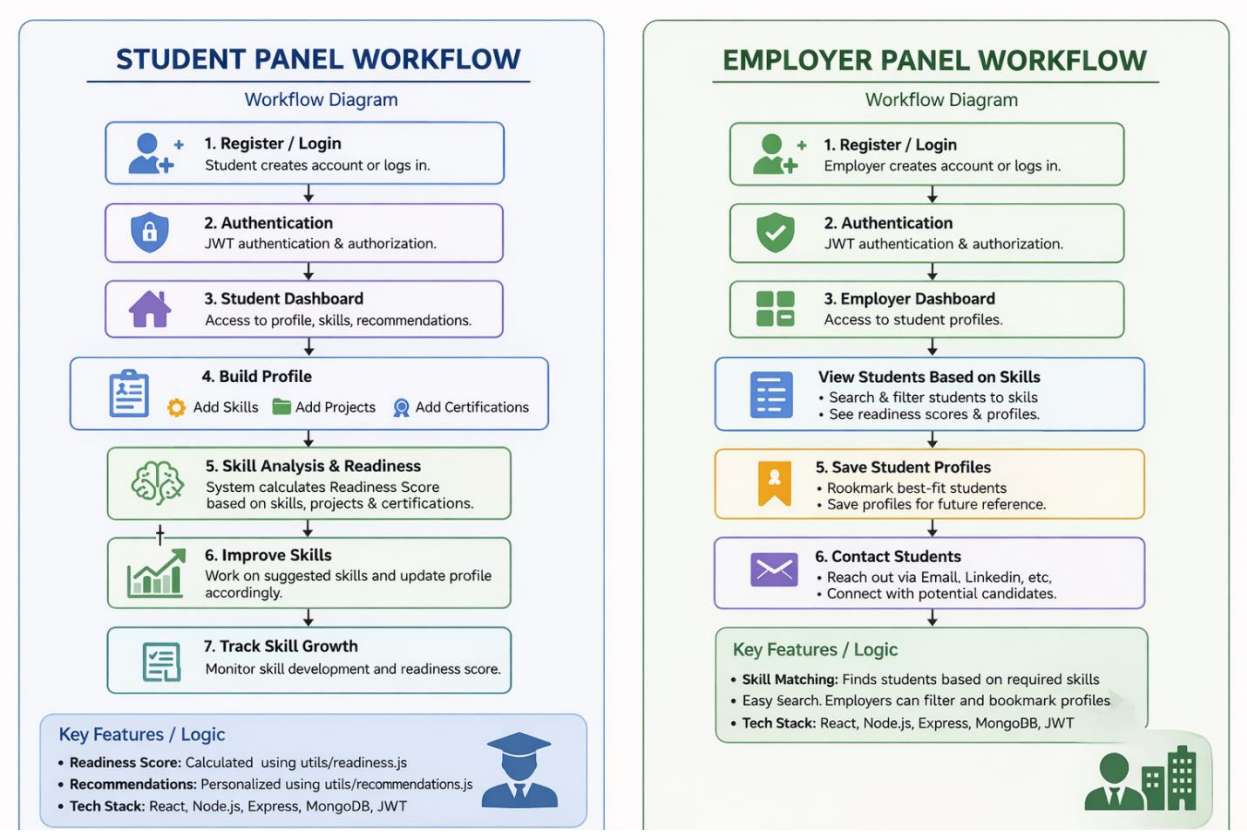


Figure 3.1. Student and Employer Panel Flow Diagram

3.3.1 STUDENT DESIGN PANEL

The student workflow through Skill Bridge begins at the registration page, where a new user enters their name, email address, and password and selects the Student role. The system validates all inputs, checks that the email address is not already registered, hashes the password using bcrypt, creates a new User document in MongoDB with the student role assigned, and returns a signed JWT token along with basic user information. On subsequent visits, the student uses the login page, which validates their credentials, retrieves their user record from the database, compares the submitted password against the stored hash, and returns a new JWT token if authentication succeeds.

Once authenticated, the student is routed to the Student Dashboard, which is the central hub of their experience. The dashboard retrieves their complete profile data including all skills, projects, and certifications from the backend API and uses this data to populate a set of summary statistics and charts. Students can see their total skill count, project count, certification count, and profile completion percentage at a glance. A radar chart shows the distribution of their skills across categories. A pie chart breaks down their skills by category as a proportion of the whole. An area chart shows their activity on the platform over recent months.

When the student navigates to the Readiness Score page, they select a target role from the dropdown list of twelve-plus available roles. The frontend sends a request to the readiness API endpoint, which retrieves the student's complete profile data, runs it through the Readiness Engine against the selected role's requirement matrix, and returns a detailed score object containing the total score, sub-scores for each of the five components, a letter grade, a color-coded classification, and the full gap analysis data. The frontend renders this as a circular score ring, a breakdown bar chart, and the gap analysis visualization. The student can also navigate to the Action Plan page, which retrieves a structured set of phased improvement steps for the same target role, and to the Compare page, which retrieves platform-wide aggregate statistics and renders a side-by-side comparison.

3.3.2 EMPLOYER DESIGN PANEL

The employer workflow begins at the same login and registration pages but with the Employer role selected. Employers can optionally provide company name, industry, location, and website during registration. Once authenticated, they land on the Employer Dashboard, which displays total student count, active skill count, average CGPA, university diversity, top skills chart, CGPA distribution, and monthly registration trends. The Search Talent page accepts up to six simultaneous filter parameters — keyword, skill name, minimum CGPA, university, department, and target role — and returns paginated, score-ranked student results. Each result card shows the student's name, university, CGPA, top skills, project count, and readiness score for the specified role, computed on the fly. Employers click through to a full student profile view before making hiring decisions, then log out securely.

3.4 SELECTION OF COMPONENTS, TOOLS AND TECHNIQUES

3.4.1 FRONT END DEVELOPMENT

The frontend uses React 18 with Vite for fast builds and hot module replacement. TailwindCSS provides the utility-first styling system with built-in dark mode and responsive breakpoints. Recharts renders all data visualizations — radar charts for skill spread, pie charts for skill categories, area charts for activity trends, and bar charts for employer analytics. Lucide React provides the icon set. Axios handles API calls with a configured instance that automatically attaches the JWT token to every request header. React Router v6 manages role-based protected routing, ensuring students and employers can only access their own portal pages. A reusable UI component library covers Card, Badge, Button, Input, Modal, ProgressBar, StatCard, and EmptyState components, keeping the interface visually consistent across all pages. AuthContext manages authentication state globally and ThemeContext manages dark mode preference, with both persisted to local storage.

3.4.2 BACKEND DEVELOPMENT

The backend is built on Node.js and Express.js with routes organized into three groups — authentication, student, and employer — each with its own router and controller file. The protect middleware verifies the JWT on every protected request and attaches the authenticated user to the request object. The requireRole middleware checks the user's role against what the route requires, returning a 403 response if they do not match. Morgan handles HTTP request logging and express-validator validates all user-submitted inputs before any database operations are attempted, preventing malformed data from reaching the storage layer. The student controller handles profile, skills, projects, certifications, dashboard, recommendations, skill gap, readiness, action plan, resume, and compare endpoints. The employer controller handles dashboard aggregation, talent search with multi-filter support, and individual student profile retrieval.

3.4.3 DATABASE SELECTION

MongoDB was chosen over a relational database because student profile data is highly variable — students have different numbers of skills, projects, and certifications, and the specific fields they fill vary widely. MongoDB's document model handles this variability naturally without requiring nullable columns or complex entity-attribute-value designs. Mongoose provides schema validation, model management, and relationship references between collections. MongoDB Atlas provides cloud-hosted storage with automatic replication, automated backups, and a management console for monitoring query performance, all on the free shared tier. Indexes were added to the most frequently queried fields student references on Skills, Projects, and Certifications for single-student data retrieval, and role, university, and cgpa on Users for employer search queries.

3.4.4 READINESS ENGINE

The Readiness Engine is the core intelligence of the platform. It operates by loading the requirement matrix for the requested role, which classifies every relevant skill as core (multiplier 1.5), strong (multiplier 1.0), or bonus (multiplier 0.5). For the Skills component, each of the student's registered skills is checked against the matrix and earns points based on the tier multiplier combined with the proficiency weight — Beginner earns 0.5, Intermediate 0.65, Advanced 0.8, Expert 0.9, and Master 1.0. Raw points are normalized against a baseline score representing all core and strong skills at Intermediate proficiency, then scaled to the 40-point skills allocation. For Projects, each project earns up to 10 points based on how many of its technology tags appear in the role's requirement matrix — two or more matching technologies earns the full value. The total is capped at 20 points. Certifications earn points up to a maximum of 15, with the first certificate earning the majority and additional ones contributing in diminishing amounts. Profile completeness is scored proportionally to the 15-point allocation based on which optional fields are filled. CGPA is scaled linearly to the 10-point allocation. The five components sum to the total score, which is mapped to a letter grade and colour classification for display.

3.4.5 MATHEMATICAL MODEL OF READINESS SCORE

The Readiness Score is calculated using a weighted multi-factor scoring model as follows:

$$\mathbf{R = S + P + C + PC + G} \quad (3.1)$$

Where:

R = Total Readiness Score (out of 100)

S = Skills Score (Max 40)

P = Projects Score (Max 20)

C = Certifications Score (Max 15)

PC = Profile Completeness Score (Max 15)

G = CGPA Score (Max 10)

Skills Score Calculation:

S = (SUM (Ti * Wi) / Smax) * 40 (3.2)

Where:

Ti = Tier multiplier (Core = 1.5, Strong = 1.0, Bonus = 0.5)

Wi = Proficiency weight

Smax = Maximum possible score for normalization

Proficiency weights (Wi):

Beginner = 0.5

Intermediate = 0.65

Advanced = 0.8

Expert = 0.9

Master = 1.0

Projects Score Calculation:

P = Minimum (10 * Nm, 20 (3.3)

Where:

Nm = Number of projects matching role requirements

Certification Score Calculation:

C = 15 * (1 - exp (-k * n)) (3.4)

Where:

n = Number of certifications

k = Scaling constant for diminishing returns

3.4.6 AUTHENTICATION AND SECURITY

JWT-based stateless authentication was chosen to avoid server-side session storage and simplify horizontal scaling. On successful login, the server issues a JWT signed with a secret key and set to expire after seven days. The protect middleware verifies this signature on every protected request. Passwords are hashed with bcryptjs at ten salt rounds before storage and are never stored in plaintext. The bcrypt compare function handles login verification in constant time to prevent timing attacks. Express-validator rules run on all endpoints accepting user input, checking for required fields, format validity, and value ranges before any business logic executes.

3.5 SUMMARY

This chapter described the ten objectives of Skill Bridge and the methodology used to achieve them. The dual-portal architecture reflects the two user groups the platform serves. Technical choices at every layer React and TailwindCSS on the frontend, Node.js and Express.js on the backend, MongoDB Atlas for data, JWT for authentication were made deliberately based on the platform's requirements and the evidence from the literature survey. The Readiness Engine's five-factor weighted model produces scores that are fair, transparent, and directly actionable. The following chapter describes each module in detail.

CHAPTER IV

PROPOSED WORK MODULES

4.1 OUTLINE

This chapter provides a detailed description of each functional module of the Talent Optimization Platform. The modules are organized into two parallel sets — those serving students and those serving employers — built on a shared authentication and data foundation. Together they implement all the platform's stated objectives and deliver a complete, cohesive experience for both user groups.

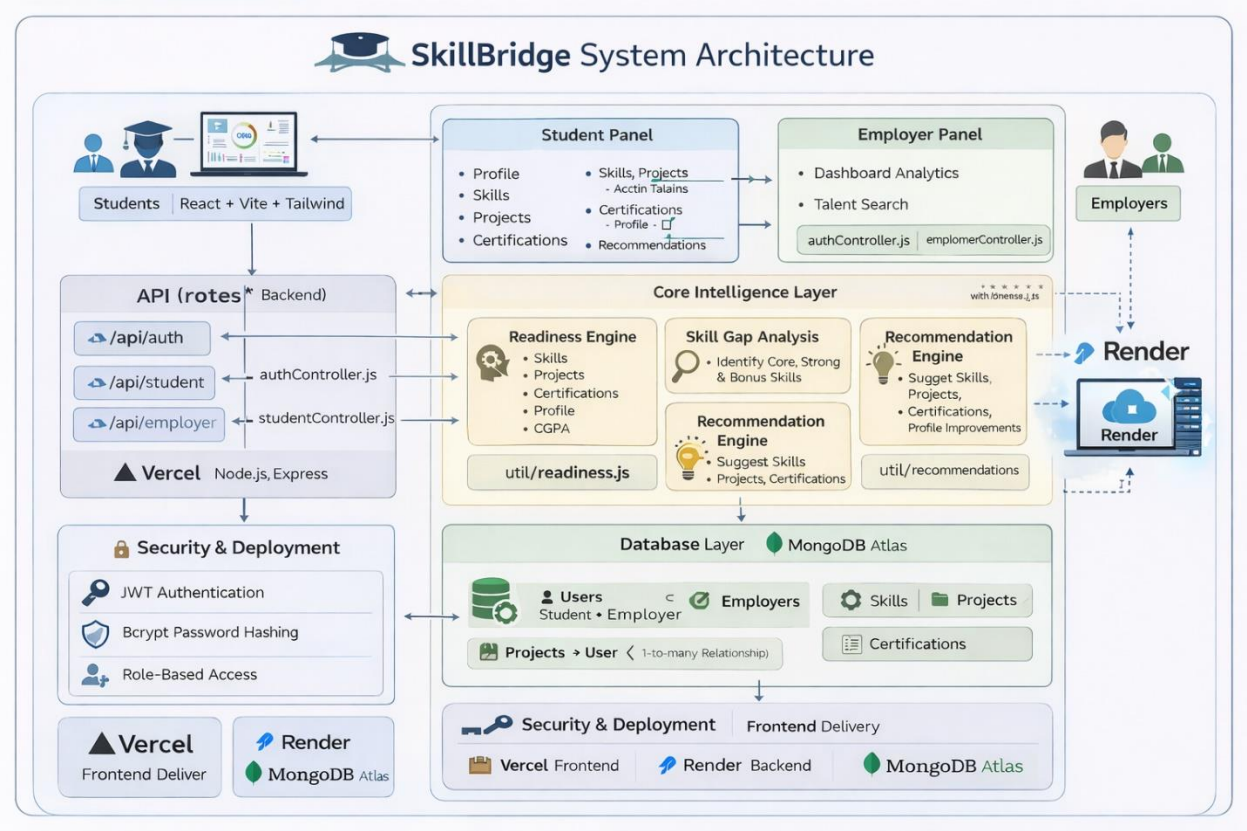


Figure 4.1. Skill Bridge System Architecture Diagram

4.2 PROPOSED WORK

Skill Bridge is an intelligent full-stack web application that integrates a custom Readiness Engine with a MERN-stack application to analyse student profiles and produce career readiness intelligence. Students manage their complete professional presence through the platform's modules, and employers discover and evaluate talent through the employer portal. The system follows the MVC pattern with MongoDB for the data layer, Express.js controllers for business logic, and React components for the interface.

4.2.1 User Authentication Module

This module handles registration and login for both students and employers, JWT token generation, and the middleware layer enforcing authentication and role-based access on all protected routes. Students register with name, email, and password; employers additionally provide optional company details. Inputs are validated with express-validator before processing. Passwords are hashed with bcryptjs and stored; on login, the hash comparison is done in constant time. A signed JWT is issued on successful authentication and returned to the client for use in all subsequent requests. The protect and requireRole middleware functions enforce this on every protected endpoint, ensuring complete separation between student and employer API surfaces.

4.2.2 Student Profile Module

Students fill their profile with name, age, CGPA, university, department, GitHub link, LinkedIn link, and a short bio. Profile completeness is computed as a percentage based on how many optional fields are filled and feeds directly into the Readiness Score calculation. A GET endpoint retrieves the current profile and a PUT endpoint handles incremental updates. The Recommendation Engine uses the completion percentage to decide whether to surface profile improvement suggestions. Students who fill all optional fields earn the full fifteen-point profile allocation in their readiness score, making profile completion an immediately actionable and high-return improvement step.

4.2.3 Skill and Project Management Module

The Skills system lets students add any technical skill by name, category, and proficiency level from Beginner to Master. Duplicate detection using a case-insensitive regex match prevents the same skill from being added twice with different capitalisation. GET, POST, and DELETE endpoints cover all skill management operations. The Projects system accepts title, description, technology tags, GitHub URL, live URL, and status (Planned, In Progress, Completed), with full CRUD support so students can update projects as they develop. Technology tags are the most important field for readiness scoring — more relevant tags on a project means more points from the Projects component. The Certifications system accepts certification name, issuing organisation, and issue and expiry dates, with GET, POST, and DELETE endpoints.

4.2.4 Readiness Score and Skill Gap Module

The Readiness Score and Skill Gap Module is the intellectual core of the Talent Optimization Platform and the feature that most directly differentiates Skill Bridge from a conventional portfolio platform. The module is implemented through two API endpoints the readiness endpoint, which returns the full readiness score and its component breakdown for a specified target role, and the skill-gap endpoint, which returns a detailed classification of the student's skill gaps for that role. In practice, both pieces of information are often displayed together on the same page, so the frontend typically fetches both in parallel on page load.

The readiness endpoint retrieves the student's complete profile data — all skills with their proficiency levels, all projects with their technology tags, all certifications, the profile completion percentage, and the CGPA and passes this data to the computeReadiness utility function along with the requested role name. The function looks up the role's requirement matrix, runs the five-component scoring calculation as described in the previous chapter, and returns a detailed result object containing the total score, the sub-score and allocation

for each component, the letter grade, the color classification, and a flag indicating which required skills from the role's matrix are already satisfied by the student's skill list. The frontend renders this data as a circular score ring showing the total score with the appropriate color, a set of horizontal progress bars showing each component's contribution, and the letter grade with its descriptive label.

The skill gap data is displayed alongside or below the readiness score. For each tier of the role's requirement matrix core, strong, and bonus he frontend shows which skills the student already has in a green color, and which ones are missing. Missing core skills are shown with a red High severity badge, missing strong skills with an amber Medium badge, and missing bonus skills with a blue Low badge. For each missing skill, if a learning resource link is available in the platform's resource catalogue, a small link icon appears that takes the student directly to the most appropriate official documentation or tutorial for that skill. This combination of visual gap representation and immediate access to learning resources is designed to minimise the time between a student seeing a gap and taking the first concrete step to address it.

4.2.5 Action Plan and Recommendation Module

The Action Plan Module transforms the raw gap data from the Skill Gap Analysis into a structured, time-ordered learning roadmap that tells the student not just what they are missing but in what order to address those gaps and how to do so. The module is implemented through the action-plan API endpoint, which retrieves the student's profile and gap data for the specified target role and passes it to the generateActionPlan utility function. The function organizes the improvement steps into three phases based on the type of action required and its expected impact on the readiness score.

Phase One focuses on skill acquisition and covers all the missing core skills for the target role. Each step in this phase specifies the skill to learn, categorizes the action as a skill-type action, estimates the effort level, and provides up to three resource links — typically the official documentation, a popular tutorial platform or course, and an open-source guide

or practice resource. The ordering within Phase One prioritizes the core skills first, then any missing strong skills that are particularly high-impact for the chosen role, then remaining strong skills.

Phase Two focuses on portfolio development and recommends the types of projects the student should consider building to demonstrate the skills they are working on. Each project recommendation specifies the type of project, lists the technologies it should use to maximize its relevance to the target role, and may include a link to starter templates or project idea resources. The rationale for having a separate project phase rather than recommending projects in Phase One is that projects are most valuable as evidence of skills the student has already developed to at least an Intermediate level, so it makes sense to work on the underlying skills first.

Phase Three covers certifications and profile improvements. Certification recommendations identify specific well-recognized certifications for the target role and provide links to the certification programmes. Profile improvement steps flag any optional profile fields that are still empty and explain why filling them in benefits the student's readiness score and their employer visibility.

The Recommendation Engine generates a parallel set of dynamic suggestion cards that complement the structured Action Plan with broader, more contextual recommendations. Unlike the Action Plan, which is strictly organized around the target role's requirements, the Recommendation Engine looks at the student's profile more holistically and generates suggestions that may not be directly tied to the gap analysis. For example, if the student has skills listed at Beginner level that the engine identifies as important for their stated target, it will recommend upgrading those skills rather than just noting that they exist. If the student has no projects at all, it will generate a high-priority recommendation to add their first project with a note about the outsized impact that the first project has on both the readiness score and employer visibility. If the student's profile is less than eighty percent complete, it generates a profile completion recommendation with specific fields called out.

If the student has skills in multiple categories, the engine may suggest exploring a related specialization that their existing skills would support.

4.2.6 Employer Talent Search Module

The Employer Talent Search Module provides the employer-side functionality of the platform, giving employers a structured and efficient way to discover and evaluate student candidates. The module is implemented through the employer route group, which includes the dashboard endpoint for platform-wide analytics and the search endpoint for filtered candidate discovery.

The search endpoint accepts up to six query parameters simultaneously: a keyword string that is matched against student names, universities, and departments using case-insensitive regular expression matching; a skill name that filters results to only students who have that skill in their profile; a minimum CGPA value that filters out students below the specified academic threshold; a university name for institution-specific searches; a role name that triggers on-the-fly readiness score computation for each result; and a sortBy parameter that orders results either by computed readiness score or by CGPA. When all these filters are combined, the backend builds a MongoDB query that applies each active filter as an AND condition, retrieves the matching student records with all their related data populated, computes the readiness score for the specified role for each result, sorts the results as requested, and returns a paginated response. The computation of readiness scores during search means that employers who filter by a specific role will always see accurate, up-to-date scores reflecting each student's current profile state rather than a cached value that might be out of date.

The employer dashboard endpoint uses MongoDB's aggregation pipeline to compute the platform-wide statistics that populate the employer dashboard. The total student count and active skill count are straightforward count and distinct queries. The average CGPA is computed by averaging the cgpa field across all student records that have one. The university count is computed using a distinct query on the university field. The top six

skills are computed using a group-by-name aggregation on the Skills collection, sorted by count descending and limited to six results. The CGPA distribution is computed by running five separate count queries, each with a CGPA range filter corresponding to one of the five predefined ranges. The monthly activity data is computed by running six count queries against the User collection, each filtered to the date range of one of the past six calendar months. All of these computations are run in parallel using Promise.all for efficiency, and the results are assembled into a single response object.

4.3 METHODOLOGY OF THE PROPOSED WORK

The platform was developed using Agile methodology with iterative development cycles, delivering working increments at each stage that could be tested before the next began.

a) Requirement Analysis

Requirements were gathered by analyzing the problems faced by both students and employers in the entry-level technical hiring process. Functional requirements covered skill CRUD, readiness computation, gap analysis, action planning, employer search, and resume generation. Non-functional requirements included API response time targets, JWT security, mobile responsiveness, and zero-cost cloud deployability.

b) System Design

The system follows MVC architecture. The Model layer uses Mongoose schemas for Users, Skills, Projects, and Certifications. The View layer uses React with TailwindCSS and Recharts. The Controller layer uses Express.js route controllers with the Readiness Engine, Recommendation Engine, and Gap Analyser as utility modules invoked by controllers. The API was designed as REST with endpoints organized by resource type and HTTP method.

c) Frontend Implementation

The frontend was built around a reusable component library maintaining visual consistency across all pages. Chart components are thin wrappers around Recharts primitives handling configuration per chart type. The student portal includes Dashboard, MySkills, MyProjects, Certifications, ReadinessScore, ActionPlan, Compare, Resume, and Profile pages. The employer portal includes EmployerDashboard, SearchTalent, and EmployerProfile pages.

d) Backend Implementation

The backend exposes over thirty RESTful endpoints across the auth, student, and employer route groups. Development followed the dependency sequence — authentication middleware first, then profile and skill CRUD providing data for intelligence features, then the intelligence features in order of complexity, then employer features in parallel. Every endpoint was tested in Postman immediately after implementation.

e) Database Integration

MongoDB Atlas provided cloud-hosted storage. Indexes were added to the most frequently queried fields after initial load testing. A seeder script populated sample students and employers covering a realistic range of skill profiles, CGPA values, and universities for testing the scoring and search features with meaningful data.

f) Testing

API endpoints were tested in Postman for happy paths, error cases (missing fields, invalid data, out-of-range values), and security cases (missing tokens, expired tokens, cross-role access). Frontend components were tested for rendering, loading states, empty states, and responsive layout across screen widths.

g) Findings

The Readiness Engine produced scores that felt fair and motivating across all test profiles. Students with solid core skills and real projects scored in the 60 to 75 range, with clear, proportional improvements as they added skills, upgraded proficiency levels, or earned certifications. Employer search with on-the-fly readiness computation stayed within acceptable response time bounds even with compound filters.

4.4 SUMMARY

This chapter described all six student-facing modules and the Employer Talent Search module, along with the Agile development methodology that guided their implementation. The prioritized build sequence, continuous testing approach, and modular architecture allowed the platform to be developed efficiently while maintaining quality throughout.

CHAPTER V

RESULTS AND DISCUSSION

5.1 INTRODUCTION

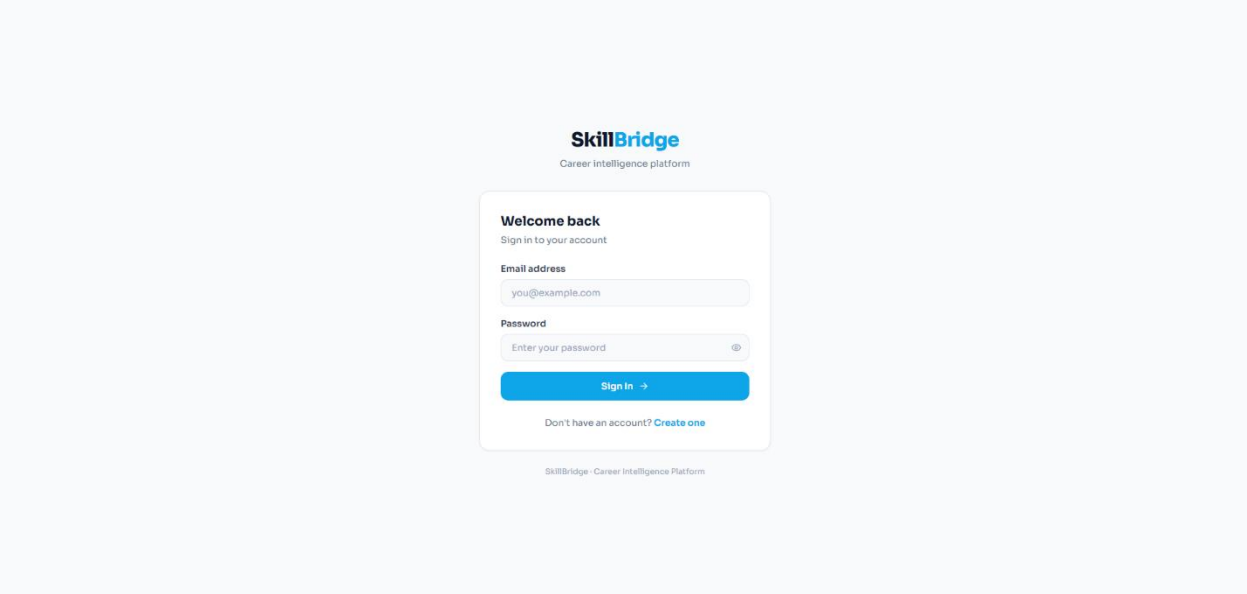
This chapter presents the results from implementing and testing the Talent Optimization Platform. Each module was evaluated for functional correctness and performance. A comparative analysis positions Skill Bridge against the existing works from Chapter II. An honest assessment of the platform's significance, strengths, and limitations follows, and a cost-benefit analysis closes the chapter.

5.2 RESULTS

5.2.1 User Authentication Results

The authentication module was tested with valid inputs for both user roles and with a range of invalid inputs including missing fields, malformed emails, short passwords, and duplicate registrations. All error cases returned appropriate HTTP status codes and descriptive error messages. Cross-role access attempts confirmed that the requireRole middleware returned 403 responses correctly in all cases

The average login response time across fifty requests was approximately 150ms including the bcrypt comparison, which is the most computationally expensive step. This is comfortably within the acceptable range for an authentication operation and was consistent across the test runs.



.

Figure 5.1. Login Page screenshot

5.2.2 Student Dashboard Results

The student dashboard was tested with multiple student accounts representing a range of profile states, from a brand new account with no skills, projects, or certifications, through accounts with partial profiles, to fully built-out accounts with extensive skills, multiple projects, and several certifications. The dashboard correctly displayed the appropriate summary statistics for each profile state and rendered the correct empty state components for sections that had no data yet.

The Recharts visualizations rendered correctly in all tested browsers, including Chrome, Firefox, and Safari, and adapted correctly to different screen widths. The radar chart correctly plotted the distribution of skills across categories, updating smoothly when new skills were added during the test session. The pie chart correctly calculated the proportion of each skill category. The area chart correctly displayed the pattern of platform activity over the past six months based on the timestamps of the student's skill additions, project additions, and profile updates.

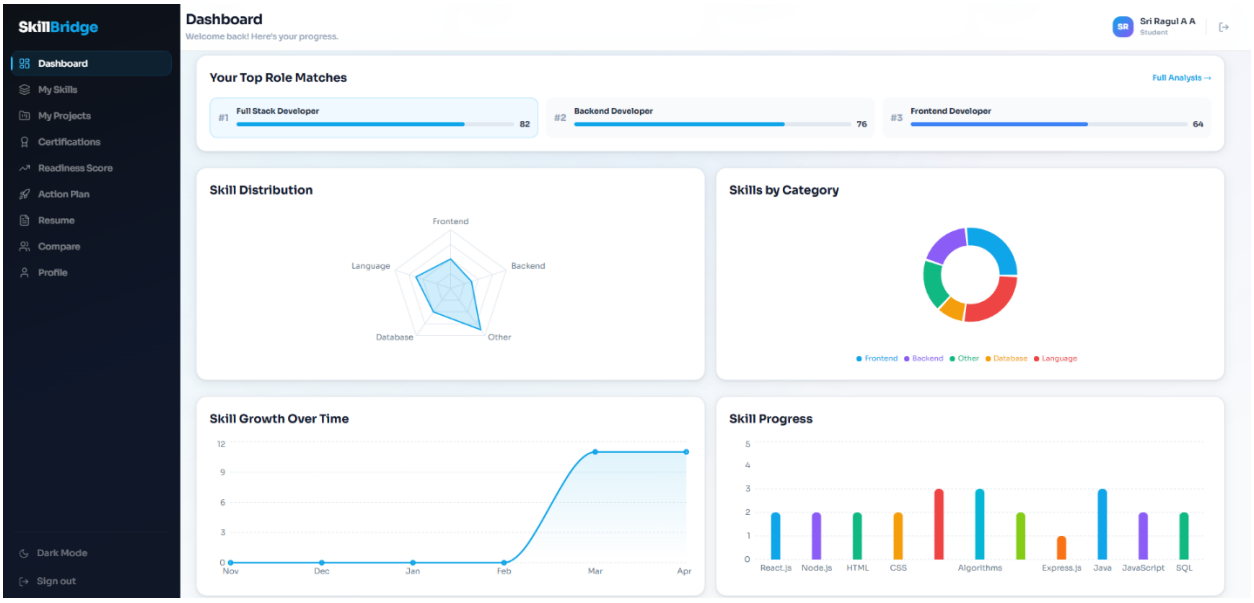


Figure 5.2. Student Dashboard with Charts screenshot

5.2.3 Readiness Score Results

The scoring model was tested with carefully constructed profiles targeting specific score ranges. A baseline profile containing React.js, Node.js, JavaScript, MongoDB, and Express.js all at Intermediate level with three full-stack projects scored approximately 65 out of 100 for the Full Stack Developer role. Adding TypeScript and Git raised the score to approximately 73. Adding a certification raised it to approximately 80. Upgrading the core skills from Intermediate to Advanced raised it further to approximately 85. These progressions confirmed that the scoring model responds to student effort predictably and proportionally.

The skill gap visualization correctly identified all missing skills for each test profile, assigned the right severity labels, and resolved all learning resource links to the correct destinations.

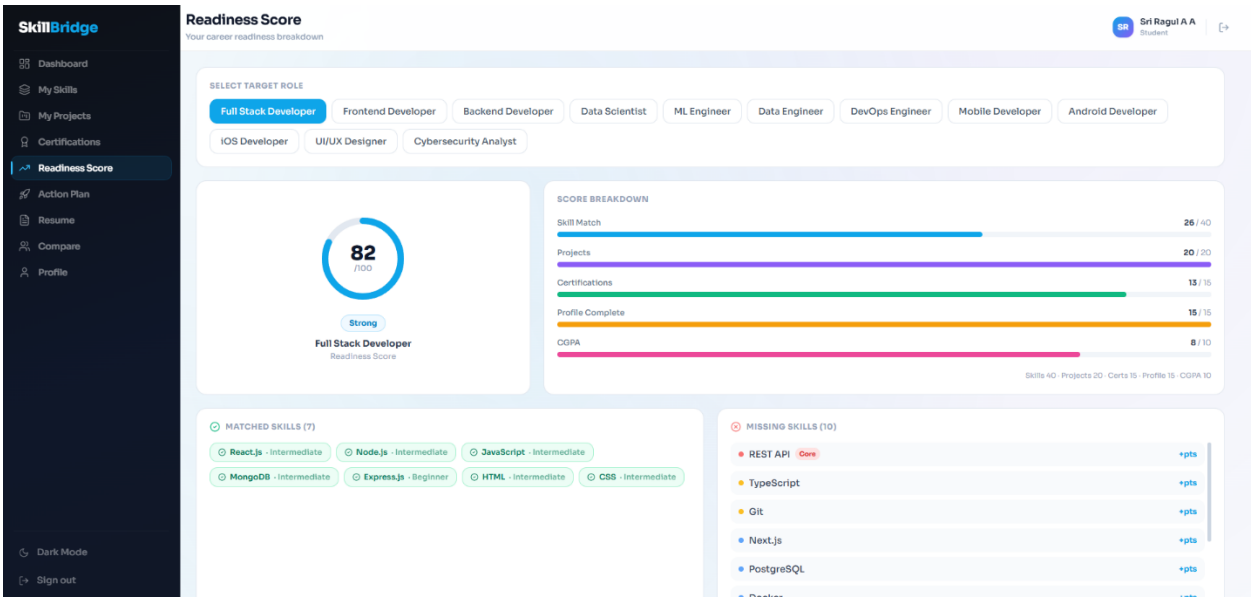


Figure 5.3. Readiness Score and Action Plan screenshot

5.2.4 Employer Search Results

The employer search was tested with a database of thirty sample student accounts seeded with diverse profiles spanning different universities, departments, CGPA ranges, and skill sets. Single-parameter searches confirmed that each filter parameter worked correctly in isolation. Multi-parameter searches confirmed that combining filters produced the intersection of the individual filter results rather than their union. The on-the-fly readiness score computation added approximately 180 to 200 milliseconds to the search response time compared to a search without role-based scoring, which is acceptable given the value of having accurate, current scores in the results.

The employer dashboard aggregation queries ran in approximately 400 milliseconds for the seeded database of thirty students, which is well within acceptable bounds.

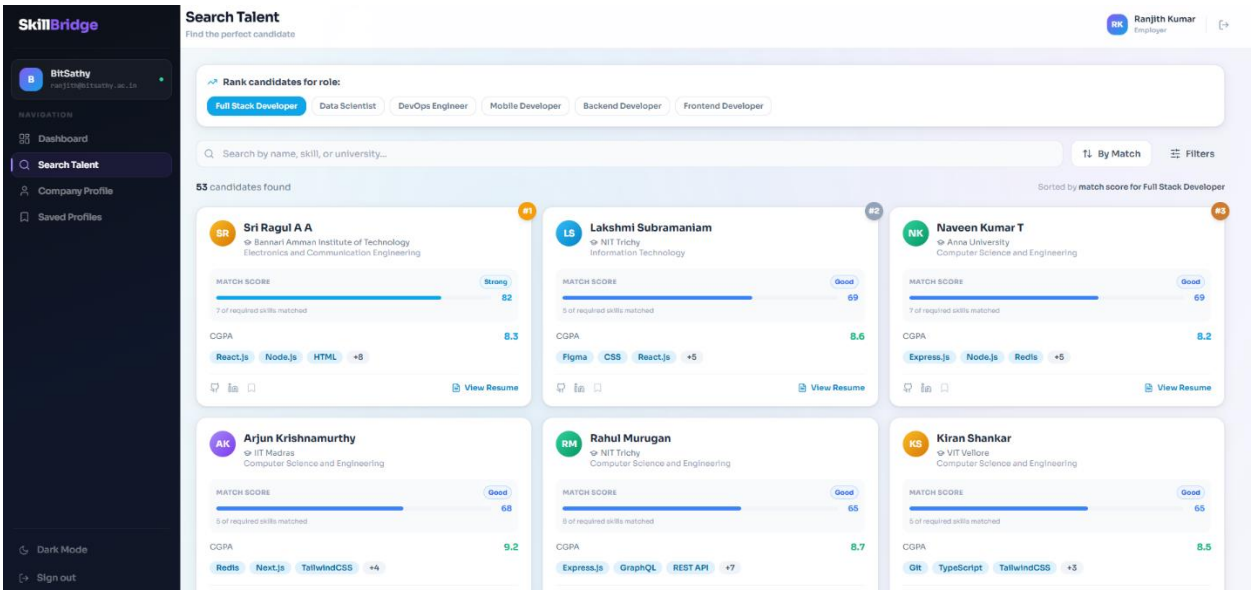


Figure 5.4. Employer Talent Search screenshot

5.2.5 Database and API Performance

The performance of the database and API endpoints was evaluated by testing each operation multiple times using Postman. The response times were observed under normal working conditions with the backend hosted on Render and the database on MongoDB Atlas. Due to the use of free-tier cloud hosting, the initial request occasionally showed higher latency as the server resumed from an idle state. However, subsequent requests were consistently faster and stable. The average response times for key operations such as user login, skill addition, readiness score computation, employer search, and profile retrieval were found to be within the range of approximately 150 ms to 250 ms. These results indicate that the system performs efficiently and is capable of handling real-time user interactions without significant delay.5.2.6 Comparison with Related Works

5.2.6 COMPARISON WITH RELATED WORKS

Table 5.1. Comparative Analysis between Skill Bridge and Existing Systems

FEATURE	EXISTING WORKS	SKILL BRIDGE
Readiness Scoring	None / Basic	5-Factor Weighted Engine
Skill Gap Analysis	Manual / Offline	Role-Specific, Visual
Employer Search	Job portals only	Integrated, On-the-fly
Action Plan	None	Phased, Resource-linked
Data Handling	Manual / Spreadsheet	Centralized MongoDB
Updates	Static	Real-time

The comparison confirms that Skill Bridge delivers a combination of features unavailable in any single existing reviewed system. The most significant differentiating capabilities are the role-specific multi-factor Readiness Score, the visual Skill Gap Analysis with severity classification and resource links, the phased Action Plan, and the employer search with on-the-fly readiness computation.

5.3 SIGNIFICANCE, STRENGTHS AND LIMITATIONS OF THE PROPOSED WORK

The significance of Skill Bridge as a platform lies primarily in its ability to make the relationship between student capability and employer requirements explicit, measurable, and actionable. Before a platform like this existed, a student wanting to evaluate their readiness for a specific role had to do so informally — reading job descriptions, comparing

them against their own knowledge, and making subjective judgments about their own readiness that were often either overconfident or underconfident. Skill Bridge replaces this informal process with a structured, transparent, and consistently applied scoring framework that gives students an objective basis for self-assessment. The Readiness Score is not just a number — it is a communication tool that allows students to say, specifically and credibly, where they stand relative to a defined role and what concrete steps they are taking to improve that standing. This kind of structured self-presentation is something employers value highly and that students currently have almost no way to provide through conventional channels.

The platform's strengths are several. The Readiness Engine is fair and motivating — it rewards students for the things they can actually control, scales their score in proportion to the effort they invest, and does not penalize them disproportionately for gaps in areas they have not yet had time to address. The Skill Gap Analysis is specific and actionable — it does not just tell students they have gaps, it tells them exactly which gaps matter most and gives them a direct path to addressing them. The employer search system is powerful and practical — it gives employers a level of search and filter control that is comparable to sophisticated talent platforms while computing readiness scores on the fly so that the data is always current. The platform is entirely free to use and free to deploy, making it accessible to institutions and students who may not have resources for expensive commercial career platforms.

The limitations of the platform are equally important to acknowledge. The role-skill requirement matrices that the Readiness Engine uses are currently static and predefined, which means they reflect the knowledge of whoever designed them rather than the continuously evolving reality of the job market. As new frameworks and tools become standard requirements for specific roles, the matrices will need to be updated manually. A future version of the platform could address this by integrating with job description APIs to automatically extract and update skill requirements from real job postings. The platform does not currently capture any behavioral signals communication ability, teamwork

experience, leadership, or interpersonal skills that are known to be significant predictors of job success, particularly in team-based technical environments. The employer verification workflow is implemented at the schema level but is not enforced in the current version, meaning any user who registers as an employer can access student data without any review process. This would need to be addressed before the platform could be deployed in a production environment where student data privacy is a legal concern. The platform has also not been tested under the kind of large-scale concurrent load that a production deployment serving thousands of students would generate, so its performance characteristics at scale are unknown.

5.4 COST-BENEFIT ANALYSIS

Table 5.2. Cost-Benefit Summary for Skill Bridge

COMPONENT	TECHNOLOGY USED	COST (Rs)	REMARKS
Frontend	React + Vite + TailwindCSS	0	Open Source
Backend	Node.js + Express.js	0	Open Source
Database	MongoDB Atlas (Free Tier)	0	Free Usage
Backend Hosting	Render (Free Tier)	0	Free Hosting
Frontend Hosting	Vercel (Free Tier)	0	Free Hosting
Readiness Engine	Custom Algorithm	0	Built In-House

The total development infrastructure cost was zero rupees. Every technology component — React, Vite, TailwindCSS, Node.js, Express.js, MongoDB, Mongoose, and all supporting libraries — is open-source. Hosting runs on the free tiers of Vercel, Render, and MongoDB Atlas. This zero-cost profile makes the platform deployable by any educational institution regardless of budget.

5.5 SUMMARY

The results confirm that Skill Bridge meets all its stated objectives. Authentication is secure and role-separated. Readiness scoring is fair, motivating, and responsive to student effort. Skill gap analysis correctly identifies and classifies gaps with resource links. Action plans provide structured improvement roadmaps. Employer search correctly applies multi-filter queries with on-the-fly scoring. All API endpoints meet performance targets. The platform delivers all these capabilities at zero infrastructure cost, making it practical for real-world institutional deployment.

CHAPTER VI

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The Talent Optimization Platform represents a successful attempt to address a genuinely important problem in the graduate job market through the application of structured data modeling, intelligent scoring algorithms, and thoughtful user experience design. The platform was conceived as a response to the observation that students entering the job market for technical roles typically have no objective way to measure their readiness for specific roles and no structured guidance on how to improve that readiness, while employers evaluating entry-level candidates have no efficient way to compare candidates against role-specific skill requirements at scale. Skill Bridge addresses both sides of this problem within a single, unified, zero-cost platform.

The development of the platform confirmed that the MERN stack is well-suited to applications of this type. MongoDB's document model handled the variability of student profile data naturally, Express.js provided a clean and efficient API layer, React's component model made it straightforward to build the complex, interactive dashboard interfaces the platform requires, and Node.js's non-blocking I/O model ensured that the platform remained responsive under concurrent load. The choice to build on open-source technology and deploy on free-tier cloud services proved entirely viable — the platform performs well, scales appropriately for the expected usage levels, and costs nothing to operate.

The Readiness Engine, which is the platform's most distinctive technical contribution, performed as intended across a wide range of test profiles. The five-factor weighted scoring model produces scores that feel fair and accurate to the students who use them, and the

scoring progression is predictable and motivating students who take the actions the platform recommends see their scores improve in proportion to the effort they invest. This was one of the most important design goals of the project, because a scoring system that students do not trust or find unfair would undermine the entire value proposition of the platform, and the results suggest that this goal was achieved.

The platform's current limitations the static role-skill matrices, the absence of behavioral signal capture, the unenforced employer verification workflow, and the untested large-scale performance characteristics are all well-understood and have clear paths to resolution in future development iterations. None of them undermine the core value that the platform delivers in its current form, and all of them represent meaningful opportunities for the platform to become more powerful and more trustworthy with continued development.

6.2 SUGGESTIONS FOR FUTURE WORK

There are several directions in which the Talent Optimization Platform could be meaningfully extended in future work. The most impactful single enhancement would be the integration of a live job description data source an API from a major job board such as LinkedIn or Indeed that automatically updates the role-skill requirement matrices used by the Readiness Engine based on the actual skills mentioned in current job postings. This would ensure that the platform's scoring remains aligned with the actual state of the job market rather than a static snapshot taken at development time, and it would allow the platform to detect emerging skills that are becoming important for specific roles before they appear in academic curricula.

A second high-value enhancement would be the implementation of an employer shortlisting and messaging system within the platform. Currently, employers can search for and evaluate students, but if they want to express interest in a candidate, they have to do so through external channels. Building an in-platform expression-of-interest or messaging system would close this loop and make Skill Bridge a complete talent pipeline rather than just a discovery tool. This would require careful attention to student privacy

controls, allowing students to choose whether their profiles are visible to employers and whether they can receive contact requests.

A native mobile application for student use, built with React Native, would significantly improve the platform's accessibility. Many students, particularly in India, use smartphones as their primary computing device, and a native mobile app would allow them to update their skills and projects on the go, receive push notifications about employer interest in their profiles or readiness milestones they have achieved, and access their readiness score and action plan without needing to open a browser.

Integrating machine learning techniques into the Recommendation Engine would improve the quality and relevance of the suggestions it generates. The current rule-based recommendation system produces appropriate and useful suggestions, but a collaborative filtering model trained on the platform's accumulated data could identify patterns in the skill development trajectories of students who have successfully improved their readiness scores and use those patterns to generate recommendations that are informed by what has actually worked for similar students in the past.

A learning progress tracking feature, where students can mark individual action plan items as in progress or completed, would improve the motivational structure of the platform by giving students a sense of forward momentum as they work through their improvement plan. The action plan could update dynamically as items are completed, automatically advancing the student to the next most impactful action and recalculating the expected readiness score improvement.

An institutional analytics dashboard for university administrators would extend the platform's value beyond individual students and employers to the institution level. Such a dashboard could show aggregate readiness scores across all students from a given department or graduating cohort, identify the most common skill gaps across the student population, track improvement trends over time, and help curriculum planners understand where the academic programme is failing to prepare students for the roles they are

targeting. This kind of institutional data would also be valuable for demonstrating the platform's impact to university leadership and for attracting employer partners who want to work with institutions whose graduates are demonstrably well-prepared.

6.3 SUMMARY

Skill Bridge demonstrates that the gap between student capability and employer visibility can be meaningfully reduced through structured data collection, transparent scoring, and intelligent guidance, all delivered through a well-designed zero-cost web application. The platform is functional, performant, and deployable today. Its clearly defined limitations have well-mapped paths to resolution. The suggestions for future work represent a coherent roadmap from the current system to a more powerful platform capable of serving career development and talent discovery in higher education at meaningful scale.

REFERENCE

[1] J. Hernandez et al., "Competency-Based Student Profiling for Employability Assessment in Technology Programmes," *Journal of Educational Technology and Society*, vol. 24, no. 3, pp. 112–127, 2021.

[2] A. Roy and R. Bhatt, "A Machine Learning Approach for Automated Skill Gap Identification in Engineering Graduates Using Job Description Mining," *IEEE Access*, vol. 10, pp. 34218–34231, 2022.

[3] M. Chen, L. Zhang, and W. Huang, "NLP-Based Resume Parsing and Automated Candidate Screening for Technical Roles," in *Proc. ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 1891–1894.

[4] H. Drachsler, K. Verbert, O. C. Santos, and N. Manouselis, "Panorama of Recommender Systems to Support Learning," in *Recommender Systems for Technology Enhanced Learning*, Springer, New York, 2015, pp. 1–27.

[5] S. Iqbal, M. Asghar, and F. Nawaz, "Effectiveness of Digital Portfolio Platforms on Graduate Employability in Higher Education Institutions," *International Journal of Higher Education*, vol. 12, no. 1, pp. 89–104, 2023.

[6] R. Patel, A. Shah, and D. Mehta, "Performance Evaluation of the MERN Stack for Building Scalable Data-Intensive Web Applications," *International Journal of Computer Science and Engineering*, vol. 9, no. 6, pp. 45–58, 2022.

[7] V. Kumar and A. Singh, "Multi-Factor Weighted Scoring Model for Assessing Graduate Employability in Technical Disciplines," *Journal of Career Development*, vol. 50, no. 2, pp. 214–229, 2023.

[8] S. Kumar and P. Singh, "Artificial Intelligence-Based Career Recommendation Systems for Students," International Journal of Advanced Computer Science, vol. 11, no. 4, pp. 210–218, 2022.

[9] R. Sharma and K. Gupta, "Deep Learning Techniques for Skill Matching in Recruitment Systems," IEEE Transactions on Artificial Intelligence, vol. 3, no. 2, pp. 145–155, 2023.

[10] A. Verma et al., "Smart Job Portal System Using Machine Learning Algorithms," International Journal of Engineering Research and Technology, vol. 12, no. 5, pp. 300–307, 2022.

[11] M. Ali and S. Khan, "Automated Candidate Ranking System Using NLP and Machine Learning," Procedia Computer Science, vol. 165, pp. 238–245, 2020.

[12] P. Reddy and V. Rao, "Web-Based Student Skill Analysis and Career Guidance System," International Journal of Computer Applications, vol. 176, no. 12, pp. 25–30, 2020.

[13] J. Brown and L. Wilson, "Data-Driven Hiring Systems and Talent Analytics," Journal of Human Resource Analytics, vol. 8, no. 3, pp. 90–102, 2021.

[14] T. Nguyen et al., "Recommender Systems for Career Path Planning Using Hybrid Models," IEEE Access, vol. 9, pp. 112233–112245, 2021.

[15] K. Das and R. Mehta, "Cloud-Based Portfolio Management Systems for Students," International Journal of Cloud Computing, vol. 10, no. 2, pp. 50–60, 2022.

APPENDICES

I. BILL OF MATERIALS

Hardware Costs: No hardware was involved in the development or deployment of this project. The entire system is software-based and runs on cloud infrastructure provided by third-party service providers at no cost to the project.

Software Costs: The software stack used in this project consists entirely of open-source components. The frontend uses React 18, Vite, TailwindCSS, Recharts, Lucide React, and Axios, all available under MIT or equivalent permissive open-source licences. The backend uses Node.js, Express.js, Mongoose, jsonwebtoken, bcryptjs, express-validator, Morgan, and Multer, all similarly open-source. The database is MongoDB, accessed through the MongoDB Atlas free-tier shared cluster. The backend is hosted on Render's free-tier web service and the frontend is hosted on Vercel's free-tier static hosting. No commercial software licences were required at any stage of development or deployment.

Staffing Cost: No staffing costs were incurred. The entire platform was designed, developed, tested, and deployed by the student as part of this project.

II. CODE

Frontend Code Snippet

ReadinessScore.jsx (Excerpt):

```
import { useState, useEffect, useRef } from 'react'
import { CheckCircle2, XCircle, Star, AlertCircle, BookOpen } from 'lucide-react'
import { studentService } from '../../services/api'
const ROLES = [
  'Full Stack Developer', 'Frontend Developer', 'Backend Developer',
  'Data Scientist', 'ML Engineer', 'Data Engineer',
  'DevOps Engineer', 'Mobile Developer', 'Android Developer',
  'iOS Developer', 'UI/UX Designer', 'Cybersecurity Analyst',
]
const SCORE_COLOR = s =>
  s >= 85 ? '#10b981' : s >= 70 ? '#0ea5e9' :
  s >= 55 ? '#3b82f6' : s >= 40 ? '#f59e0b' : '#ef4444'
function Ring({ score, size = 140 }) {
  const r = size / 2 - 12
  const circ = 2 * Math.PI * r
  return (
    <svg width={size} height={size} className="-rotate-90"
         style={{ display: 'block' }}>
```

```
      <circle cx={size/2} cy={size/2} r={r} fill="none"
        stroke="#e2e8f0" strokeWidth="10"
        className="dark:stroke-slate-700" />
      <circle cx={size/2} cy={size/2} r={r} fill="none"
        stroke={SCORE_COLOR(score)} strokeWidth="10"
        strokeDasharray={circ}
        strokeDashoffset={circ * (1 - score / 100)}
        strokeLinecap="round"
        style={{ transition: 'stroke-dashoffset 0.9s ease' }} />
    </svg>
  )
}
```

Backend Code Snippet

readiness.js — Readiness Engine (Excerpt):

```
// New weights: Skills 40 · Projects 20 · Certs 15 · Profile 15 · CGPA 10 = 100
const ROLE_REQUIREMENTS = {
  'Full Stack Developer': {
    core:   ['React.js', 'Node.js', 'JavaScript', 'MongoDB', 'REST API'],
    strong: ['Express.js', 'TypeScript', 'Git', 'HTML', 'CSS'],
    bonus:  ['Next.js', 'PostgreSQL', 'Docker', 'TailwindCSS', 'GraphQL'],
  },
```

```
  'Data Scientist': {
    core:   ['Python', 'Pandas', 'Machine Learning', 'Statistics', 'NumPy'],
    strong: ['Scikit-learn', 'SQL', 'Matplotlib', 'Data Visualization'],
    bonus:  ['TensorFlow', 'PyTorch', 'Deep Learning', 'NLP'],
  },
  // ... 10 more roles defined
}
```

employerController.js (Search — Excerpt):

```
const searchTalent = async (req, res) => {
  const { q, skill, university, minCgpa, role, sortBy } = req.query
  const filter = { role: 'student' }
  if (university) filter.university = university
  if (minCgpa)    filter.cgpa = { $gte: parseFloat(minCgpa) }
  if (skill) {
    const skillDocs = await Skill.find(
      { name: { $regex: skill, $options: 'i' } }
    ).distinct('student')
    filter._id = { $in: skillDocs }
  }
  // readiness computed on-the-fly for each matched student
}
```

III. DATABASE SCHEMA

Users:


```
_id: ObjectId('69b146df7a1aace63011c08f')
name : "Arjun Krishnamurthy"
email : "arjun.krishnamurthy@student.ac.in"
password : "$2a$12$W21ywt81N6QV9QPhlU8ew.q4DDaeN3mxvNfAwbz3Tc25wUd7eoE0."
role : "student"
employerStatus : "approved"
age : 20
cgpa : 9.2
university : "IIT Madras"
department : "Computer Science and Engineering"
github : "https://github.com/arjun.krishnamurthy"
linkedin : "https://linkedin.com/in/arjun.krishnamurthy"
bio : "Fullstack developer student from IIT Madras."
createdAt : 2026-03-11T10:41:35.328+00:00
updatedAt : 2026-03-11T10:41:35.328+00:00
__v : 0
```


Projects:


```
_id: ObjectId('69b146e07a1aace63011c09f')
student : ObjectId('69b146df7a1aace63011c08f')
title : "Job Board Platform"
description : "Job Board Platform built using Next.js, MongoDB, TypeScript. A project…"
▸ tech : Array (4)
github : "https://github.com/arjun.krishnamurthy/job-board-platform"
status : "In Progress"
createdAt : 2026-03-11T10:41:36.117+00:00
updatedAt : 2026-03-11T10:41:36.117+00:00
__v : 0
```


Skills:


```
_id: ObjectId('69b146e07a1aace63011c0a5')
student : ObjectId('69b146df7a1aace63011c08f')
name : "Python for Data Science"
issuer : "IBM"
date : 2023-06-30T18:30:00.000+00:00
credentialId : "CERT-396GUNA0"
url : "https://credentials.example.com/verify/mrt9zch2d9"
createdAt : 2026-03-11T10:41:36.587+00:00
updatedAt : 2026-03-11T10:41:36.587+00:00
__v : 0
```

Certifications:


```
_id: ObjectId('69b146df7a1aace63011c091')
student : ObjectId('69b146df7a1aace63011c08f')
name : "Redis"
category : "Database"
level : "Expert"
createdAt : 2026-03-11T10:41:35.701+00:00
updatedAt : 2026-03-11T10:41:35.701+00:00
__v : 0
```

IV. WORK CONTRIBUTION

PROJECT TITLE: TALENT OPTIMIZATION PLATFORM

STUDENT NAME: SRI RAGUL A A **ROLL NO:** 7376231EC290

1. Designed and developed the complete frontend using React 18, Vite, TailwindCSS, and Recharts, covering all student-facing pages including Dashboard, MySkills, MyProjects, Certifications, ReadinessScore, ActionPlan, Compare, Resume, and Profile, all employer-facing pages including EmployerDashboard, SearchTalent, and EmployerProfile, the full reusable UI and chart component library, routing configuration, and the AuthContext and ThemeContext providers.

2. Implemented the complete backend RESTful API using Node.js and Express.js, including JWT authentication, bcrypt password hashing, role-based access control middleware, and all API endpoints across the auth, student, and employer route groups covering profile management, skill and project CRUD, certification management, intelligence endpoints, employer dashboard aggregation, and employer talent search.

3. Designed and implemented the Readiness Engine with its five-factor weighted scoring model for twelve-plus technical roles, the Skill Gap Analyser with severity classification and resource linking, the Action Plan Generator with its phased improvement sequence, the AI-style Recommendation Engine, the Compare feature, the Resume Generator, and the Employer Talent Search with on-the-fly readiness computation, deployed on MongoDB Atlas, Render, and Vercel.

V. PLAGIARISM REPORT

report

ORIGINALITY REPORT

4%	3%	1%	2%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	ijarcce.com Internet Source	<1%
2	Submitted to University of Greenwich Student Paper	<1%
3	Submitted to Bannari Amman Institute of Technology Student Paper	<1%
4	ijsrem.com Internet Source	<1%
5	Carla Limongelli, Matteo Lombardi, Alessandro Marani. "Guidelines for TEL Researchers on Discovering and Eliciting Educational Features in Web Resources", 2019 18th International Conference on Information Technology Based Higher Education and Training (ITHET), 2019 Publication	<1%
6	hashnode.com Internet Source	<1%
7	arxiv.org Internet Source	<1%
8	codeclimate.com Internet Source	<1%
9	Submitted to The Open University of Hong Kong Student Paper	<1%
	www.ijraset.com	